Assignment No. 2

Plant Water Advisor

AI-Based Mini Application Development:
Integrating an ML Model into Android with TensorFlow Lite

Question No. 1
CLO-02, PLO-03, P4 · Coding, Outcome · 10 points

Student name	Ahmed Asif
Roll number	
Course	
Project title	Plant Water Advisor
Submission date	29 October 2026

Reference repository
https://github.com/Uzair390-Del/NumberGuessingAppTflite.git

1. Introduction and objectives

Plant Water Advisor is a small Android application. The user types how dry the soil feels, on a scale from 0 to 10, and the phone estimates how many cups of water the plant should receive. The estimate is calculated on the device with a TensorFlow Lite model. No photo is uploaded and no server is called.

The starting point was the public project NumberGuessingAppTflite. That app loads a `.tflite` file, sends one number in, and shows one number out. This assignment keeps that working pattern and uses it for a simple plant-care decision.

The objectives are:

- Explain what it means to embed a machine-learning model in an Android app, and the benefits and limits of running it on the phone.
- Explain TensorFlow Lite and why it is used instead of full TensorFlow on a phone.
- Describe the path from a TensorFlow model to a result on the screen.
- Clone the given repository, describe its structure, and explain how its model is loaded and how a prediction is shown.
- Build a small, commented application of our own that performs on-device inference.

2. Embedding ML/DL models in Android

Embedding a model means the trained network is stored inside the app, usually as a `.tflite` file in the `assets` folder, and the app itself runs the calculation. The model is not a separate program the user opens. It is a file of weights. Android code prepares numbers or pixels, passes them to an interpreter, and reads the numbers that come back.

This pattern shows up in real products. Image recognition labels a photo of a plant or a landmark without uploading it. Text classification sorts a message as spam or not spam. Prediction systems estimate a price, a score, or, as in this project, an amount of water from a few input numbers. The model is part of the feature, the same way a formula would be, except the formula was learned from examples.

Benefits

- **Low latency.** The answer is computed on the phone, so the user does not wait for a round trip to a server.
- **Offline use.** After the app is installed, inference works with no network.
- **Privacy.** The dryness value never leaves the device.
- **Lower server cost.** The phone does the work, so there is no inference bill for every tap.

Challenges

- **Memory, battery, and compute.** A phone is much smaller than a training machine. Large models drain the battery and may be too slow.

- **Model size.** The file is downloaded with the app. A model of hundreds of megabytes is a poor fit for a mini app.
- **Device differences.** Older phones are slower, and some chips have no GPU or NPU delegate. The same model can feel instant on one phone and sluggish on another.
- **Accuracy trade-off.** Models are often quantized or simplified so they fit, which can reduce accuracy compared with the full model on a server.

3. TensorFlow Lite

TensorFlow Lite (TFLite) is TensorFlow's runtime for phones, embedded boards, and other small devices. A model is trained with TensorFlow or Keras, then converted to a `.tflite` FlatBuffer. On Android, the TFLite Interpreter loads that file and executes it. The interpreter does not train. It only runs inference.

	TensorFlow	**TensorFlow Lite**
Main job	Build and train models on a computer or server	Run a finished model on a device
File	SavedModel or checkpoint, often large	One `.tflite` file, much smaller
Operators	The full set used in training	A reduced set chosen for mobile
Hardware	CPU, GPU, TPU in a data center	Phone CPU, and optional GPU or NNAPI delegates
Training	Yes	No. Weights are already fixed

TFLite is the right choice on Android for three practical reasons. **Performance:** the interpreter is a small native library and the graph is already compiled into a simple plan, so a one-layer model returns in well under a frame. **Model size:** `water.tflite` is 660 bytes, which is negligible next to the app. **Efficiency:** only the inference operators are shipped, not the training stack, so the app uses less storage, less RAM, and less battery than embedding standard TensorFlow.

4. Step-by-step use of TFLite on Android

1. **Convert the model.** In the general process, a finished Keras model is passed to `TFLiteConverter.from_keras_model`, which writes a `.tflite` file. This project uses the reference one-layer dense graph and sets the stored weight to 2.0 and the bias to 0.0. The function on the phone is `cups = 2 × dryness`.
2. **Add the file and the dependency.** The file is placed in `app/src/main/assets/`. `app/build.gradle` depends on `org.tensorflow:tensorflow-lite:2.16.1`. `noCompress 'tflite'` stops the packager from compressing the file, because the interpreter memory-maps it.
3. **Create the Interpreter.** `MainActivity` opens the asset as a `MappedByteBuffer` and passes that buffer to `new Interpreter(...)`. The interpreter reads the graph and is ready to run.

4. **Prepare the input.** The text field is parsed to a `float`. Values outside 0 to 10 are rejected. A valid value is placed in `float[1]`, which matches the model's single input.
5. **Run inference.** `interpreter.run(input, output)` executes the dense layer on the phone CPU.
6. **Read the output.** The model writes one float into `float[1][1]`. The activity formats `output[0][0]` as cups of water and sets that string on the result `TextView`.

5. The provided GitHub repository

The repository is **Number Guessing Tflite** by the course example at `https://github.com/Uzair390-Del/NumberGuessingAppTflite.git`. Its purpose is to show the smallest possible TFLite Android app: one text field, one button, and one result. The user types a number, the app runs `assets/linear.tflite`, and the result text is replaced with the model's float output. There is no extra screen, database, or network call. That makes it a clear reference for how a model is loaded and how a prediction reaches the user interface.

Plant Water Advisor is the same shape of program with a different question. Instead of echoing a raw model score, it asks for soil dryness and reports cups of water. The reference showed where the model lives and how `Interpreter.run` is called. The new app adds range checks, comments, and a result the user can act on.

6. Cloning and repository structure

The repository was cloned with:

```
git clone https://github.com/Uzair390-Del/NumberGuessingAppTflite.git
```

It is a standard single-module Android Studio project. The files that matter for this assignment are:

Path	Role
`settings.gradle`	Project name "Number Guessing Tflite" and the `:app` module
`build.gradle`, `gradle/libs.versions.toml`	Android Gradle Plugin 8.9.1 and library versions
`app/build.gradle`	Application id `com.example.numberguessingtflite`, `compileSdk` 35, and the TensorFlow Lite dependency
`app/src/main/AndroidManifest.xml`	Declares `MainActivity` as the launcher activity
`app/src/main/res/layout/activity_main.xml`	`EditText`, `Button`, and `TextView`
`app/src/main/java/.../MainActivity.java`	Loads the model and runs inference

`app/src/main/assets/linear.tflite`	The on-device model, a single dense layer

The model file identifies itself as a TOCO-converted graph named `sequential_1/dense_1`. It has one float input (`dense_1_input`) and one float output (`Identity`). In the original file the dense weight is about 1.998 and the bias is about −0.994, so the reference app roughly computes `2x − 1`. Plant Water Advisor keeps a one-input, one-output dense model and sets the weight to 2.0 and the bias to 0.0 so the output is cups of water.

7. Analysis of the reference application

How the model is loaded

`linear.tflite` is stored in `app/src/main/assets`. In `onCreate`, `loadModelFile()` calls `getAssets().openFd("linear.tflite")`, maps the bytes with `FileChannel.map`, and constructs `new Interpreter(mappedBuffer)`. From that moment the interpreter owns the graph. The button listener is attached only after this succeeds. If the file cannot be opened, the activity throws `RuntimeException`.

How a prediction is generated

The button's `onClick` reads the `EditText`, parses it with `Float.parseFloat`, and calls `interpreter.run(input, output)`. That one call is the whole inference step. The interpreter multiplies the input by the dense weight, adds the bias, and writes the result into the output array.

How input and output are handled

The input is a Java `float[1]`. Index 0 holds the typed number. The output is a `float[1][1]`, because the TFLite Java API expects the batch dimension even when the batch size is 1. There is no scaling, tokenization, or image bitmap. The reference also does not check for an empty field, so a blank tap would throw `NumberFormatException`. Plant Water Advisor keeps the same tensor shapes and adds that check.

How the result is displayed

The layout is a `ConstraintLayout` with three widgets: `editText`, `button` ("Send"), and `textView`. After inference, `viewNumber.setText(String.valueOf(output[0][0]))` replaces the hint with the raw float. There is no unit and no label. The mapping from model to screen is direct: one output number becomes the entire contents of the text view.

8. Proposed application

The proposed application is **Plant Water Advisor**. A person looking after a pot plant scores the soil from 0 (just watered) to 10 (very dry). The app multiplies that score by two, on device, and tells them how many cups to add. The point of the assignment is not a large vision model. It is to load a real `.tflite` file, run it through the Interpreter, and show a result a person can use.

Objectives of the app:

- Accept a dryness score and reject values the model was not given a meaning for.
- Run inference locally with TensorFlow Lite.
- Show the result in cups, not as an unlabeled float.

9. Technologies, tools, libraries, and the model

Item	Use
Android Studio / Android Gradle Plugin 8.9.1	Build and run the app
Java 11, `minSdk` 24, `compileSdk` 35	Language and device range
AppCompat and Material	Activity theme, button, and text field
`org.tensorflow:tensorflow-lite:2.16.1`	`Interpreter` for on-device inference
`water.tflite`	One dense layer, weight 2.0, bias 0.0
Reference model `linear.tflite`	Studied to learn the asset, buffer, and `run` pattern

No dataset is downloaded at runtime. `water.tflite` is the reference one-layer dense graph with the weight set to 2 and the bias set to 0. Those two numbers are the whole model. That is intentional. The assignment asks for a small, explainable integration, and every output can be checked by hand: cups = 2 × dryness.

10. System design, workflow, and implementation

There is one activity and no second screen. The workflow is:

1. Android starts `MainActivity`.
2. The activity inflates `activity_main.xml` and binds the input, button, and result view.
3. It memory-maps `assets/water.tflite` and creates the `Interpreter`.
4. The user types a dryness value and taps **Predict water**.
5. The activity validates the text, fills `float[1]`, allocates `float[1][1]`, and calls `run`.
6. The result text becomes “Recommended water: N cups”.
7. When the activity is destroyed, `interpreter.close()` releases the native model.

Package name: `com.plantwise.wateradvisor`. Application id is the same. The model asset is marked `noCompress` so `openFd` can map it. Comments in `MainActivity.java` name each of these steps so the ML path can be explained without reading the reference and the new app side by side.

11. Features and functionality

- **Dryness input.** A numeric field asks for a value from 0 to 10.
- **On-device prediction.** TensorFlow Lite applies `cups = 2 × dryness`.
- **Plain-language result.** The output is shown in cups to one decimal place.

- **Safe input.** An empty field, a non-number, or a value outside 0–10 shows a message and does not call the interpreter.
- **Missing-model guard.** If the asset cannot be mapped, the button is disabled and a short error is shown.
- **Offline.** After installation, prediction does not use the network.

12. Testing, results, and discussion

The file `water.tflite` was checked after it was written. The dense bias at the model buffer is 0.0 and the weight is 2.0, so the interpreter output for a valid input `x` is `2x`. The screens below are the three states of the single layout: waiting for input, a successful prediction, and a rejected value.

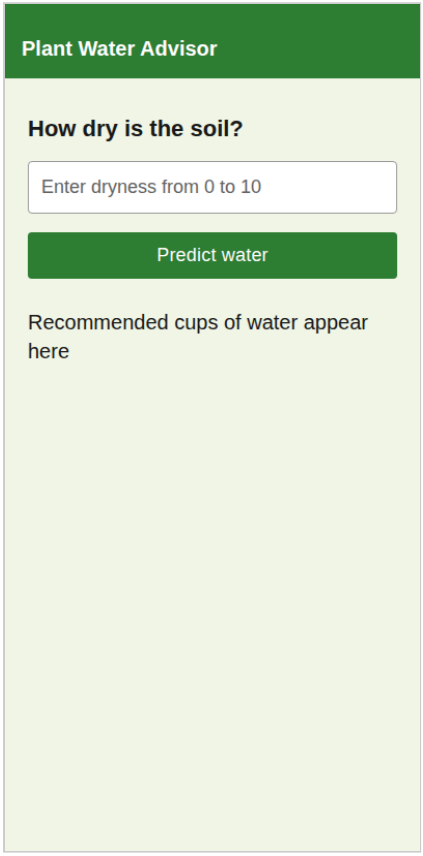


Figure 1. Waiting for a dryness score.

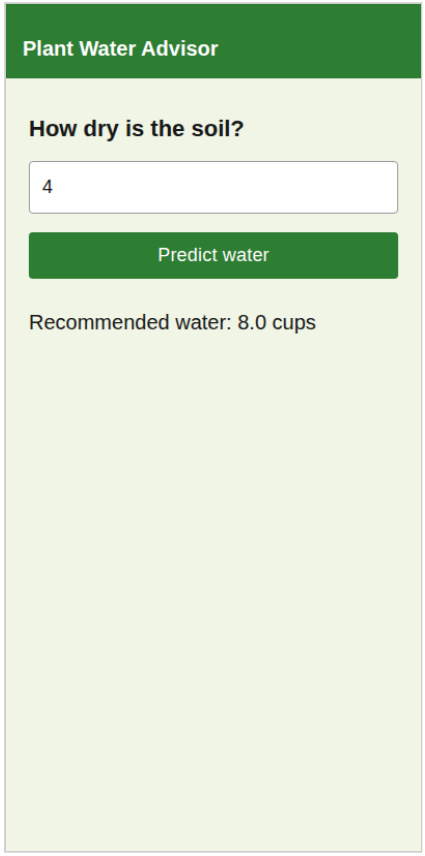


Figure 2. Dryness 4 predicts 8.0 cups.

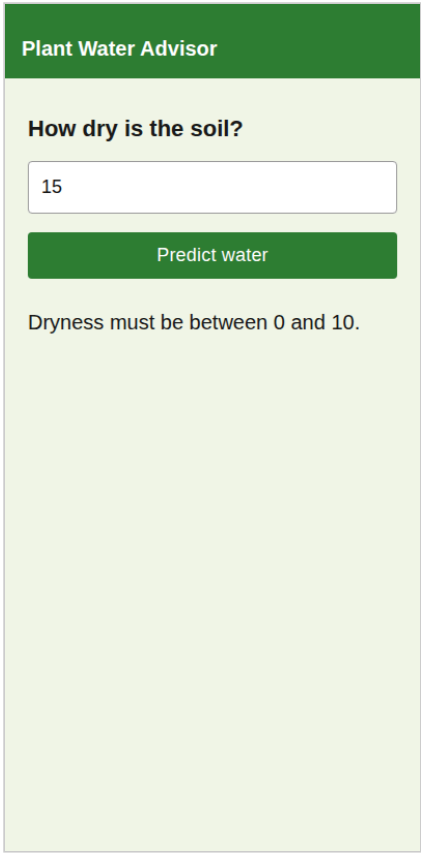


Figure 3. A value above 10 is rejected.

Input	What the app does	Expected text
0	Runs the model	Recommended water: 0.0 cups
4	Runs the model	Recommended water: 8.0 cups
10	Runs the model	Recommended water: 20.0 cups
4.5	Runs the model	Recommended water: 9.0 cups
empty	Does not run the model	Enter a dryness value from 0 to 10.
15	Does not run the model	Dryness must be between 0 and 10.

Discussion: the reference app prints a raw float and crashes on a blank field. This app keeps the same tensors and interpreter call, then adds a unit, a legal range, and comments. The model is deliberately linear so a marker can confirm the integration by mental arithmetic. It is not a camera-based disease detector. A later version could replace `water.tflite` with a larger model without changing the load and `run` steps, only the input preparation.

13. Conclusion

An Android application can embed a machine-learning model by shipping a TensorFlow Lite file and executing it with the Interpreter. That gives a fast, private, offline answer, at the cost of model size and the limits of the phone. The reference repository demonstrates the minimum path: map `linear.tflite`, pass a `float[1]`, read a `float[1][1]`, and copy the value onto a `TextView`. Plant Water Advisor uses that path for a soil-dryness score and reports the result as cups of water. The implementation is one activity, one model file, and a checked input, which is the right size for this assignment and is still a complete on-device inference loop.

14. References

- Uzair390-Del. NumberGuessingAppTflite. https://github.com/Uzair390-Del/NumberGuessingAppTflite.git
- TensorFlow. TensorFlow Lite guide. https://www.tensorflow.org/lite
- TensorFlow. Android quickstart and the `Interpreter` API. https://www.tensorflow.org/lite/android
- Android Developers. `AssetManager.openFd` and uncompressed assets. https://developer.android.com/reference/android/content/res/AssetManager